<u>ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ</u>

Катедра „Компютърни системи“

КУРСОВ ПРОЕКТ
ПО БАЗИ ОТ ДАННИ

Студент: Алекс Александров Огнянов

ФАК. №: 121224052 **Група:** 43а

Тема №12

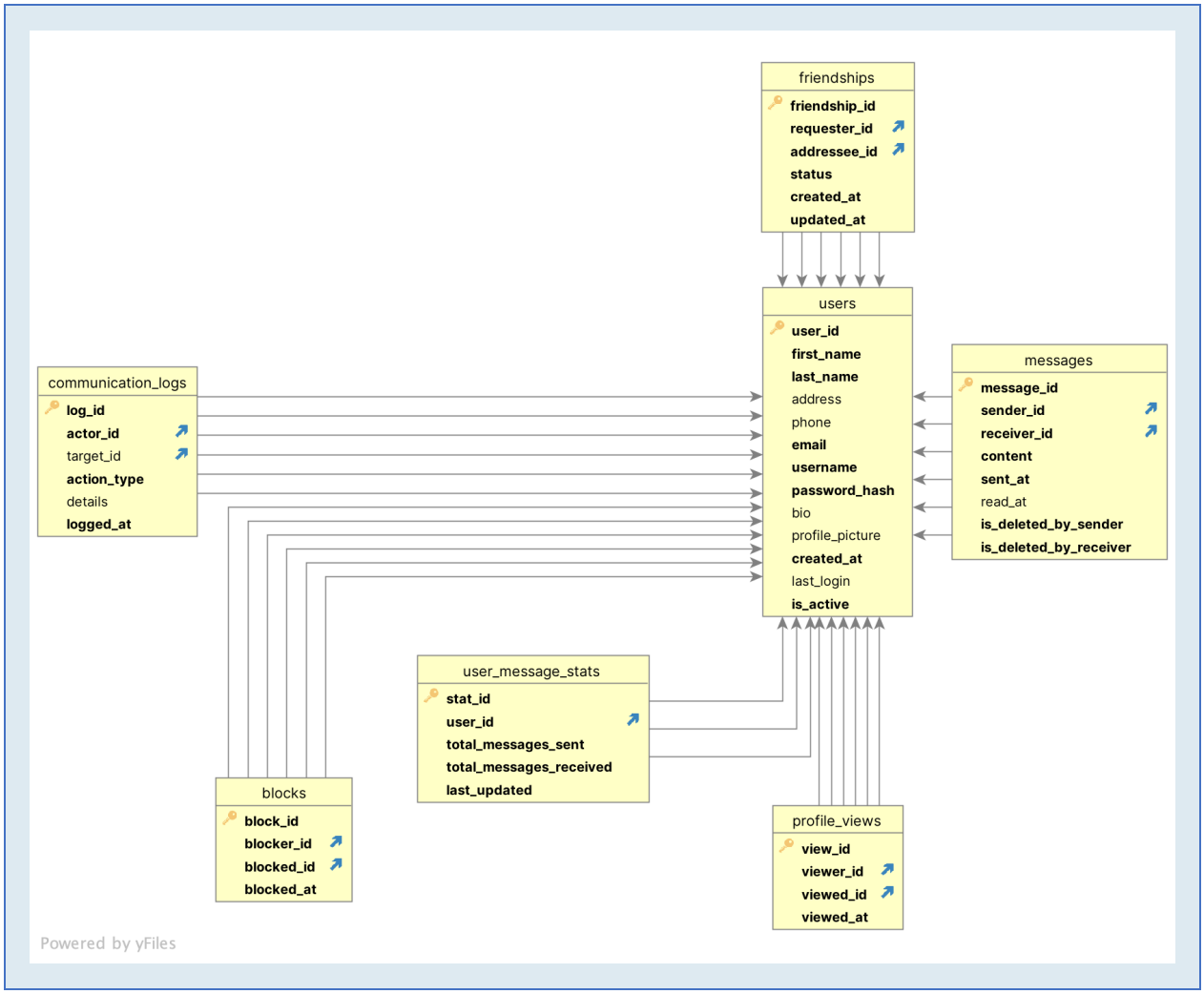
Да се проектира база данни за система за комуникация на един потребител с друг. В базата се пазят данни за клиентите – имена, адрес, телефон, и т.н., както и потребителско име и парола. Пазят се и пълни „логове“ на комуникацията. Да се пази информация за свързаност между клиентите, комуникация, добавяне в приятелска листа, блокиране, преглед на профила и т.н.

1. Да се проектира база от данни и да се представи ER диаграма със съответни CREATE TABLE заявки за средата MySQL.
2. Напишете заявка, в която демонстрирате SELECT с ограничаващо условие по избор.
3. Напишете заявка, в която използвате агрегатна функция и GROUP BY по ваш избор.
4. Напишете заявка, в която демонстрирате INNER JOIN по ваш избор.
5. Напишете заявка, в която демонстрирате OUTER JOIN по ваш избор.
6. Напишете заявка, в която демонстрирате вложен SELECT по ваш избор.
7. Напишете заявка, в която демонстрирате едновременно JOIN и агрегатна функция.
8. Създайте тригер по ваш избор.
9. Създайте процедура, в която демонстрирате използване на курсор.

Вашата работа трябва да включва: задание, ER-диаграма, CREATE TABLE заявки, всички останали заявки, решения на задачите от 2 до 9 и резултатите от тях.

1. Да се проектира база от данни и да се представи ER диаграма със съответни CREATE TABLE заявки за средата MySQL:

Основните обекти, за които трябва да съхраняваме информация, са: users (потребители), messages (директни съобщения), friendships (приятелски връзки и заявки), blocks (блокирани потребители), profile_views (история на прегледани профили), communication_logs (пълни логове на комуникацията) и user_message_stats (агрегирана статистика, 1:1 с users). Таблицата users съхранява личните данни (имена, адрес, телефон, имейл), потребителско име, хеш на паролата и флаг за активност. Таблицата messages свързва подател и получател чрез FOREIGN KEY към users и съдържа флагове за изтриване и прочитане. Таблицата friendships има ENUM статус (pending/accepted/rejected) и UNIQUE ограничение за двойка потребители. За проектирането на базата ще използваме модела ER-диаграма (Entity Relationship Diagram):



Заявките, с които създаваме базата данни и таблиците са:

```sql
-- ============================================================
-- ТЕМА №12 - Система за комуникация между потребители
-- Задача 1: CREATE TABLE заявки (MySQL)
-- ============================================================

CREATE DATABASE IF NOT EXISTS messaging_system
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;

USE messaging_system;

-- ------------------------------------------------------------
-- Таблица: users
-- Съдържа личните данни и акаунт информацията на потребителите
-- ------------------------------------------------------------
CREATE TABLE IF NOT EXISTS users (
    user_id          INT             AUTO_INCREMENT PRIMARY KEY,
    first_name       VARCHAR(50)     NOT NULL,
    last_name        VARCHAR(50)     NOT NULL,
    address          VARCHAR(255),
    phone            VARCHAR(20),
    email            VARCHAR(100)    NOT NULL UNIQUE,
    username         VARCHAR(50)     NOT NULL UNIQUE,
    password_hash    VARCHAR(255)    NOT NULL,
    bio              TEXT,
    profile_picture  VARCHAR(500),
    created_at       DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP,
    last_login       DATETIME,
    is_active        TINYINT(1)      NOT NULL DEFAULT 1
);

-- ------------------------------------------------------------
-- Таблица: messages
-- Директни съобщения между двама потребители
-- ------------------------------------------------------------
CREATE TABLE IF NOT EXISTS messages (
    message_id               INT          AUTO_INCREMENT PRIMARY KEY,
    sender_id                INT          NOT NULL,
    receiver_id              INT          NOT NULL,
    content                  TEXT         NOT NULL,
    sent_at                  DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
    read_at                  DATETIME,
    is_deleted_by_sender     TINYINT(1)   NOT NULL DEFAULT 0,
    is_deleted_by_receiver   TINYINT(1)   NOT NULL DEFAULT 0,
    FOREIGN KEY (sender_id)   REFERENCES users(user_id) ON DELETE CASCADE,
    FOREIGN KEY (receiver_id) REFERENCES users(user_id) ON DELETE CASCADE
);

-- ------------------------------------------------------------
-- Таблица: friendships
-- Заявки за приятелство и установени приятелски връзки
-- ------------------------------------------------------------
CREATE TABLE IF NOT EXISTS friendships (
    friendship_id   INT     AUTO_INCREMENT PRIMARY KEY,
    requester_id    INT     NOT NULL,
    addressee_id    INT     NOT NULL,
    status          ENUM('pending', 'accepted', 'rejected') NOT NULL DEFAULT 'pending',
    created_at      DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at      DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (requester_id) REFERENCES users(user_id) ON DELETE CASCADE,
    FOREIGN KEY (addressee_id) REFERENCES users(user_id) ON DELETE CASCADE,
    UNIQUE KEY uq_friendship (requester_id, addressee_id)
);

-- ------------------------------------------------------------
-- Таблица: blocks
```

```
-- Блокирани потребители
-- ------------------------------------------------------------
CREATE TABLE IF NOT EXISTS blocks (
    block_id    INT      AUTO_INCREMENT PRIMARY KEY,
    blocker_id  INT      NOT NULL,
    blocked_id  INT      NOT NULL,
    blocked_at  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (blocker_id) REFERENCES users(user_id) ON DELETE CASCADE,
    FOREIGN KEY (blocked_id) REFERENCES users(user_id) ON DELETE CASCADE,
    UNIQUE KEY uq_block (blocker_id, blocked_id)
);

-- ------------------------------------------------------------
-- Таблица: profile_views
-- История на прегледаните профили
-- ------------------------------------------------------------
CREATE TABLE IF NOT EXISTS profile_views (
    view_id    INT      AUTO_INCREMENT PRIMARY KEY,
    viewer_id  INT      NOT NULL,
    viewed_id  INT      NOT NULL,
    viewed_at  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (viewer_id) REFERENCES users(user_id) ON DELETE CASCADE,
    FOREIGN KEY (viewed_id) REFERENCES users(user_id) ON DELETE CASCADE
);

-- ------------------------------------------------------------
-- Таблица: communication_logs
-- Пълни логове на всички комуникационни събития
-- ------------------------------------------------------------
CREATE TABLE IF NOT EXISTS communication_logs (
    log_id       INT          AUTO_INCREMENT PRIMARY KEY,
    actor_id     INT          NOT NULL,
    target_id    INT,
    action_type  ENUM(
                    'message_sent',
                    'message_read',
                    'friend_request_sent',
                    'friend_request_accepted',
                    'friend_request_rejected',
                    'user_blocked',
                    'user_unblocked',
                    'profile_viewed'
                 ) NOT NULL,
    details      TEXT,
    logged_at    DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (actor_id)  REFERENCES users(user_id) ON DELETE CASCADE,
    FOREIGN KEY (target_id) REFERENCES users(user_id) ON DELETE SET NULL
);

-- ------------------------------------------------------------
-- Таблица: user_message_stats
-- Агрегирана статистика за изпратени/получени съобщения (1:1 с users)
-- ------------------------------------------------------------
CREATE TABLE IF NOT EXISTS user_message_stats (
    stat_id                 INT      AUTO_INCREMENT PRIMARY KEY,
    user_id                 INT      NOT NULL UNIQUE,
    total_messages_sent     INT      NOT NULL DEFAULT 0,
    total_messages_received INT      NOT NULL DEFAULT 0,
    last_updated            DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE
);

-- ------------------------------------------------------------
```

Добавяме и тестови данни в таблиците:

```
-- Примерни данни
-- ------------------------------------------------------------
INSERT INTO users (first_name, last_name, address, phone, email, username, password_hash,
bio) VALUES
('Иван',    'Петров',   'ул. Витоша 1, София',       '0888111222', 'ivan@example.com',
'ivan_p',    SHA2('pass1234', 256), 'Обичам планината.'),
('Мария',   'Иванова',  'бул. Христо Ботев 5, Пловдив','0877333444','maria@example.com',
'maria_i',   SHA2('secr3t',  256), 'Програмист и геймър.'),
('Георги',  'Стоянов',  'ул. Оборище 12, Варна',     '0866555666', 'georgi@example.com',
'gosho99',   SHA2('qwerty',  256), NULL),
('Петя',    'Димитрова','ул. Граф Игнатиев 3, София', '0855777888', 'petya@example.com',
'petya_d',   SHA2('abc123',  256), 'Обичам котки.'),
('Стефан',  'Николов',  'ул. Rakovski 20, Бургас',   '0844999000', 'stefan@example.com',
'stefan_n',  SHA2('mypass',  256), 'Музикант.');

INSERT INTO messages (sender_id, receiver_id, content, read_at) VALUES
(1, 2, 'Здравей Мария! Как си?',                     NOW()),
(2, 1, 'Здравей Иване! Добре съм, благодаря!',        NOW()),
(1, 3, 'Георги, ще излизаш ли утре?',                 NULL),
(3, 1, 'Да, ще съм там.',                             NOW()),
(4, 2, 'Мария, видя ли новия проект?',                NOW()),
(2, 4, 'Да, много е интересен!',                      NULL),
(1, 2, 'Ще се видим ли тази седмица?',                NULL),
(5, 1, 'Иване, имаш ли малко свободно време?',        NOW()),
(1, 5, 'Разбира се! Кажи.',                           NOW()),
(3, 4, 'Петя, поздрави!',                             NULL);

INSERT INTO friendships (requester_id, addressee_id, status) VALUES
(1, 2, 'accepted'),
(1, 3, 'accepted'),
(4, 1, 'accepted'),
(5, 1, 'accepted'),
(2, 5, 'pending'),
(3, 4, 'rejected');

INSERT INTO blocks (blocker_id, blocked_id) VALUES
(2, 3),
(4, 5);

INSERT INTO profile_views (viewer_id, viewed_id) VALUES
(1, 2), (1, 3), (2, 1), (3, 1), (4, 2), (5, 1), (1, 4), (2, 5);

INSERT INTO communication_logs (actor_id, target_id, action_type, details) VALUES
(1, 2, 'message_sent',             'message_id=1'),
(2, 1, 'message_sent',             'message_id=2'),
(1, 2, 'friend_request_sent',      NULL),
(2, 1, 'friend_request_accepted',  NULL),
(2, 3, 'user_blocked',             NULL),
(1, 3, 'message_sent',             'message_id=3'),
(4, 5, 'user_blocked',             NULL),
(1, 2, 'profile_viewed',           NULL),
(2, 1, 'message_read',             'message_id=1');

INSERT INTO user_message_stats (user_id, total_messages_sent, total_messages_received)
VALUES
(1, 4, 2),
(2, 3, 3),
(3, 2, 1),
(4, 1, 2),
(5, 1, 2);
```

Задача 2. Напишете заявка, в която демонстрирате SELECT с ограничаващо условие по избор – ще изведем всички активни потребители (is_active = 1), регистрирани след 01.01.2020 г., сортирани по дата на регистрация в низходящ ред:

```
-- ============================================================
-- ТЕМА №12 - Система за комуникация между потребители
-- Задача 2: SELECT с ограничаващо условие
-- ============================================================
-- Извежда всички активни потребители, регистрирани след 2020 г.,
-- чиито профили са прегледани поне веднъж, сортирани по дата.
-- ============================================================

USE messaging_system;

SELECT
    user_id,
    CONCAT(first_name, ' ', last_name) AS full_name,
    username,
    email,
    phone,
    created_at
FROM users
WHERE
    is_active = 1
    AND created_at >= '2020-01-01'
ORDER BY created_at DESC;
```

user_id	full_name	username	email	phone	created_at
1	Иван Петров	ivan_p	ivan@example.com	0888111222	2026-03-25 06:22:27
2	Мария Иванова	maria_i	maria@example.com	0877333444	2026-03-25 06:22:27
3	Георги Стоянов	gosho99	georgi@example.com	0866555666	2026-03-25 06:22:27
4	Петя Димитрова	petya_d	petya@example.com	0855777888	2026-03-25 06:22:27
5	Стефан Николов	stefan_n	stefan@example.com	0844999000	2026-03-25 06:22:27

Задача 3. Напишете заявка, в която използвате агрегатна функция и GROUP BY по ваш избор – ще изведем броя на изпратените съобщения за всеки потребител, заедно с датата на първото и последното изпратено съобщение. Показваме само потребители с повече от 1 съобщение (HAVING клауза):

```
-- ============================================================
-- ТЕМА №12 - Система за комуникация между потребители
-- Задача 3: Агрегатна функция и GROUP BY
-- ============================================================
-- Брой изпратени съобщения на потребител.
-- Показва само тези, изпратили повече от 1 съобщение.
-- ============================================================

USE messaging_system;
```

```sql
SELECT
    u.user_id,
    CONCAT(u.first_name, ' ', u.last_name) AS full_name,
    u.username,
    COUNT(m.message_id)  AS total_sent,
    MIN(m.sent_at)       AS first_message_at,
    MAX(m.sent_at)       AS last_message_at
FROM users u
JOIN messages m ON m.sender_id = u.user_id
GROUP BY u.user_id, u.first_name, u.last_name, u.username
HAVING COUNT(m.message_id) > 1
ORDER BY total_sent DESC;
```

user_id	full_name	username	total_sent	first_message_at	last_message_at
1	Иван Петров	ivan_p	4	2026-03-25 06:22:27	2026-03-25 06:22:27
2	Мария Иванова	maria_i	2	2026-03-25 06:22:27	2026-03-25 06:22:27
3	Георги Стоянов	gosho99	2	2026-03-25 06:22:27	2026-03-25 06:22:27

Задача 4. Напишете заявка, в която демонстрирате INNER JOIN по ваш избор – ще напишем заявка, с която ще изведем всички съобщения заедно с имената и потребителските имена на подателя и получателя. INNER JOIN е използван два пъти – веднъж за подателя и веднъж за получателя. Извежда се преглед на съдържанието (60 символа) и статусът дали съобщението е прочетено:

```
-- ============================================================
-- ТЕМА №12 - Система за комуникация между потребители
-- Задача 4: INNER JOIN
-- ============================================================
-- Извежда всички съобщения заедно с имената на подателя
-- и получателя (само съобщения между двама съществуващи
-- потребители - INNER JOIN гарантира това).
-- ============================================================

USE messaging_system;

SELECT
    m.message_id,
    CONCAT(s.first_name, ' ', s.last_name) AS sender_name,
    s.username                              AS sender_username,
    CONCAT(r.first_name, ' ', r.last_name) AS receiver_name,
    r.username                              AS receiver_username,
    LEFT(m.content, 60)                    AS message_preview,
    m.sent_at,
    IF(m.read_at IS NOT NULL, 'Прочетено', 'Непрочетено') AS read_status
FROM messages m
INNER JOIN users s ON s.user_id = m.sender_id
INNER JOIN users r ON r.user_id = m.receiver_id
ORDER BY m.sent_at DESC;
```

message_id	sender_name	sender_username	receiver_name	receiver_username	message_preview	sent_at	read_status
1	Иван Петров	ivan_p	Мария Иванова	maria_i	Здравей Мария! Как си?	2026-03-25 06:22:27	Прочетено
3	Иван Петров	ivan_p	Георги Стоянов	gosho99	Георги, ще излизаш ли утре?	2026-03-25 06:22:27	Непрочетено
7	Иван Петров	ivan_p	Мария Иванова	maria_i	Ще се видим ли тази седмица?	2026-03-25 06:22:27	Непрочетено
9	Иван Петров	ivan_p	Стефан Николов	stefan_n	Разбира се! Кажи.	2026-03-25 06:22:27	Прочетено
2	Мария Иванова	maria_i	Иван Петров	ivan_p	Здравей Иване! Добре съм, благодаря!	2026-03-25 06:22:27	Прочетено
6	Мария Иванова	maria_i	Петя Димитрова	petya_d	Да, много е интересен!	2026-03-25 06:22:27	Непрочетено
4	Георги Стоянов	gosho99	Иван Петров	ivan_p	Да, ще съм там.	2026-03-25 06:22:27	Прочетено
10	Георги Стоянов	gosho99	Петя Димитрова	petya_d	Петя, поздрави!	2026-03-25 06:22:27	Непрочетено
5	Петя Димитрова	petya_d	Мария Иванова	maria_i	Мария, видя ли новия проект?	2026-03-25 06:22:27	Прочетено
8	Стефан Николов	stefan_n	Иван Петров	ivan_p	Иване, имаш ли малко свободно време?	2026-03-25 06:22:27	Прочетено

Задача 5. Напишете заявка, в която демонстрирате OUTER JOIN по ваш избор – ще напишем заявка, с която ще изведем всички потребители и броя на получените от тях съобщения. Използваме LEFT OUTER JOIN, за да се включват и потребители без нито едно получено съобщение (при тях COUNT ще покаже 0):

```
-- ============================================================
-- ТЕМА №12 - Система за комуникация между потребители
-- Задача 5: OUTER JOIN
-- ============================================================
-- Извежда всички потребители и броя на получените съобщения.
-- LEFT JOIN гарантира, че потребители БЕЗ получени съобщения
-- също се показват (с 0).
```

```sql
-- ==============================================================

USE messaging_system;

SELECT
    u.user_id,
    CONCAT(u.first_name, ' ', u.last_name) AS full_name,
    u.username,
    COUNT(m.message_id) AS received_messages
FROM users u
LEFT JOIN messages m
    ON m.receiver_id = u.user_id
    AND m.is_deleted_by_receiver = 0
GROUP BY u.user_id, u.first_name, u.last_name, u.username
ORDER BY received_messages DESC;
```

user_id	full_name	username	received_messag...
1	Иван Петров	ivan_p	3
2	Мария Иванова	maria_i	3
4	Петя Димитрова	petya_d	2
5	Стефан Николов	stefan_n	2
3	Георги Стоянов	gosho99	1

Задача 6. Напишете заявка, в която демонстрирате вложен SELECT по ваш избор – ще създадем заявка, с която ще изведем потребителите, изпратили повече съобщения от средния брой изпратени съобщения. Информацията за средния брой се извлича чрез вложена подзаявка (подзаявка в подзаявка):

```
-- ============================================================
-- ТЕМА №12 - Система за комуникация между потребители
-- Задача 6: Вложен SELECT (подзаявка)
-- ============================================================
-- Извежда потребителите, които са изпратили повече съобщения
-- от средния брой изпратени съобщения за всички потребители.
-- ============================================================

USE messaging_system;

SELECT
    u.user_id,
    CONCAT(u.first_name, ' ', u.last_name) AS full_name,
    u.username,
    (
        SELECT COUNT(*)
        FROM messages m
        WHERE m.sender_id = u.user_id
    ) AS messages_sent
FROM users u
WHERE (
    SELECT COUNT(*)
    FROM messages m
    WHERE m.sender_id = u.user_id
) > (
    SELECT AVG(msg_count)
    FROM (
        SELECT COUNT(*) AS msg_count
        FROM messages
        GROUP BY sender_id
    ) AS per_user_counts
)
ORDER BY messages_sent DESC;
```

user_id	full_name	username	messages_sent
1	Иван Петров	ivan_p	4
3	Георги Стоянов	gosho99	3

Задача 7. Напишете заявка, в която демонстрирате едновременно JOIN и агрегатна функция – за всяка двойка потребители с приета заявка за приятелство (статус 'accepted') ще изведем общия брой разменени съобщения между тях. Използват се три JOIN-а (friendships → users ×2, messages) и COUNT агрегатна функция:

```
-- ============================================================
```

```sql
-- ТЕМА №12 – Система за комуникация между потребители
-- Задача 7: едновременно JOIN и агрегатна функция
-- ==============================================================
-- За всяка двойка приятели (статус 'accepted') показва
-- общия брой съобщения, разменени между тях.
-- ==============================================================

USE messaging_system;

SELECT
    CONCAT(u1.first_name, ' ', u1.last_name) AS user1_name,
    u1.username                              AS user1_username,
    CONCAT(u2.first_name, ' ', u2.last_name) AS user2_name,
    u2.username                              AS user2_username,
    COUNT(m.message_id)                      AS total_messages_exchanged
FROM friendships f
INNER JOIN users u1 ON u1.user_id = f.requester_id
INNER JOIN users u2 ON u2.user_id = f.addressee_id
LEFT JOIN messages m
    ON (m.sender_id   = f.requester_id AND m.receiver_id = f.addressee_id)
    OR (m.sender_id   = f.addressee_id AND m.receiver_id = f.requester_id)
WHERE f.status = 'accepted'
GROUP BY f.friendship_id, u1.user_id, u1.first_name, u1.last_name, u1.username,
         u2.user_id, u2.first_name, u2.last_name, u2.username
ORDER BY total_messages_exchanged DESC;
```

Resets all sorted columns

user1_name	user1_username	user2_name	user2_username	total_messages_exchang...
	_p	Мария Иванова	maria_i	3
Иван Петров	ivan_p	Георги Стоянов	gosho99	2
Стефан Николов	stefan_n	Иван Петров	ivan_p	2
Петя Димитрова	petya_d	Иван Петров	ivan_p	0

Задача 8. Създайте тригер по ваш избор – ще създадем два тригера. Тригерът trg_after_message_insert се изпълнява AFTER INSERT в таблица messages и автоматично: (1) добавя запис в communication_logs и (2) обновява user_message_stats за подателя и получателя. Тригерът trg_after_friendship_update се изпълнява AFTER UPDATE на friendships и записва в логовете при приемане или отхвърляне на заявка за приятелство:

```
-- ============================================================
-- ТЕМА №12 - Система за комуникация между потребители
-- Задача 8: Тригер
-- ============================================================
-- Тригер 1: след INSERT в messages автоматично се добавя
--   запис в communication_logs и се обновява user_message_stats.
--
-- Тригер 2: след UPDATE на friendships (при приемане/отхвърляне)
--   се добавя запис в communication_logs.
-- ============================================================

USE messaging_system;

DELIMITER $$

-- ------------------------------------------------------------
-- Тригер: trg_after_message_insert
-- Логва изпращане на съобщение и обновява статистиката.
-- ------------------------------------------------------------
DROP TRIGGER IF EXISTS trg_after_message_insert$$

CREATE TRIGGER trg_after_message_insert
AFTER INSERT ON messages
FOR EACH ROW
BEGIN
    -- Лог в communication_logs
    INSERT INTO communication_logs (actor_id, target_id, action_type, details)
    VALUES (
        NEW.sender_id,
        NEW.receiver_id,
        'message_sent',
        CONCAT('message_id=', NEW.message_id)
    );

    -- Обнови статистиката за подателя
    INSERT INTO user_message_stats (user_id, total_messages_sent,
total_messages_received)
    VALUES (NEW.sender_id, 1, 0)
    ON DUPLICATE KEY UPDATE total_messages_sent = total_messages_sent + 1;

    -- Обнови статистиката за получателя
    INSERT INTO user_message_stats (user_id, total_messages_sent,
total_messages_received)
    VALUES (NEW.receiver_id, 0, 1)
    ON DUPLICATE KEY UPDATE total_messages_received = total_messages_received + 1;
END$$

-- ------------------------------------------------------------
-- Тригер: trg_after_friendship_update
-- Логва приемане или отхвърляне на заявка за приятелство.
-- ------------------------------------------------------------
DROP TRIGGER IF EXISTS trg_after_friendship_update$$

CREATE TRIGGER trg_after_friendship_update
AFTER UPDATE ON friendships
```

```
FOR EACH ROW
BEGIN
    IF NEW.status = 'accepted' AND OLD.status != 'accepted' THEN
        INSERT INTO communication_logs (actor_id, target_id, action_type, details)
        VALUES (
            NEW.addressee_id,
            NEW.requester_id,
            'friend_request_accepted',
            CONCAT('friendship_id=', NEW.friendship_id)
        );
    ELSEIF NEW.status = 'rejected' AND OLD.status != 'rejected' THEN
        INSERT INTO communication_logs (actor_id, target_id, action_type, details)
        VALUES (
            NEW.addressee_id,
            NEW.requester_id,
            'friend_request_rejected',
            CONCAT('friendship_id=', NEW.friendship_id)
        );
    END IF;
END$$

DELIMITER ;

-- ------------------------------------------------------------
-- Демонстрация на тригерите
-- ------------------------------------------------------------

-- Изпращане на ново съобщение → тригерът записва лог и обновява stats
INSERT INTO messages (sender_id, receiver_id, content)
VALUES (3, 5, 'Стефане, поздрави!');

-- Приемане на заявка за приятелство → тригерът записва лог
UPDATE friendships
SET status = 'accepted'
WHERE requester_id = 2 AND addressee_id = 5;

-- Проверка на логовете
SELECT * FROM communication_logs ORDER BY logged_at DESC LIMIT 5;

-- Проверка на статистиката
SELECT u.username, s.total_messages_sent, s.total_messages_received
FROM user_message_stats s
JOIN users u ON u.user_id = s.user_id
ORDER BY s.total_messages_sent DESC;
```

Тестваме тригерите, като изпращаме ново съобщение и след това приемаме заявка за приятелство. Проверяваме резултатите:

log_id	actor_id	target_id	action_type	details	logged_at
12	3	5	message_sent	message_id=12	2026-03-25 07:11:22
10	3	5	message_sent	message_id=11	2026-03-25 06:23:34
11	5	2	friend_request_accepted	friendship_id=5	2026-03-25 06:23:34
9	2	1	message_read	message_id=1	2026-03-25 06:22:27
8	1	2	profile_viewed	NULL	2026-03-25 06:22:27
NULL	NULL	NULL	NULL	NULL	NULL

username	total_messages_s...	total_messages_receiv...
ivan_p	4	3
gosho99	4	1
maria_i	2	3
petya_d	1	2
stefan_n	1	3

Задача 9. Създайте процедура, в която демонстрирате използване на курсор – ще създадем процедура sp_sync_message_stats, която обхожда всички активни потребители чрез курсор (CURSOR FOR SELECT user_id FROM users WHERE is_active = 1). За всеки потребител процедурата изчислява реалния брой изпратени и получени съобщения от таблица messages и обновява user_message_stats чрез INSERT ... ON DUPLICATE KEY UPDATE. Накрая се извежда обобщен отчет:

```
-- ============================================================
-- ТЕМА №12 - Система за комуникация между потребители
-- Задача 9: Процедура с курсор
-- ============================================================
-- Процедурата обхожда всички потребители чрез курсор и за
-- всеки синхронизира таблицата user_message_stats с реалния
-- брой изпратени и получени съобщения от таблица messages.
-- Накрая извежда обобщен отчет.
-- ============================================================

USE messaging_system;

DELIMITER $$

DROP PROCEDURE IF EXISTS sp_sync_message_stats$$

CREATE PROCEDURE sp_sync_message_stats()
BEGIN
    -- Променливи за курсора
    DECLARE v_user_id       INT;
    DECLARE v_sent_count    INT;
    DECLARE v_recv_count    INT;
    DECLARE v_done          TINYINT DEFAULT 0;

    -- Курсор върху всички потребители
    DECLARE cur_users CURSOR FOR
        SELECT user_id FROM users WHERE is_active = 1;

    -- Handler за края на курсора
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = 1;

    OPEN cur_users;

    user_loop: LOOP
        FETCH cur_users INTO v_user_id;

        IF v_done THEN
            LEAVE user_loop;
        END IF;

        -- Изчисли реалния брой изпратени съобщения
        SELECT COUNT(*) INTO v_sent_count
        FROM messages
        WHERE sender_id = v_user_id;

        -- Изчисли реалния брой получени съобщения
        SELECT COUNT(*) INTO v_recv_count
        FROM messages
        WHERE receiver_id = v_user_id;

        -- Актуализирай или вмъкни запис в user_message_stats
        INSERT INTO user_message_stats
            (user_id, total_messages_sent, total_messages_received)
```

```
        VALUES
            (v_user_id, v_sent_count, v_recv_count)
        ON DUPLICATE KEY UPDATE
            total_messages_sent     = v_sent_count,
            total_messages_received = v_recv_count,
            last_updated            = NOW();

    END LOOP;

    CLOSE cur_users;

    -- Отчет след синхронизацията
    SELECT
        u.user_id,
        CONCAT(u.first_name, ' ', u.last_name) AS full_name,
        u.username,
        s.total_messages_sent,
        s.total_messages_received,
        (s.total_messages_sent + s.total_messages_received) AS total_activity,
        s.last_updated
    FROM user_message_stats s
    JOIN users u ON u.user_id = s.user_id
    ORDER BY total_activity DESC;
END$$

DELIMITER ;

-- ------------------------------------------------------------
-- Изпълнение на процедурата
-- ------------------------------------------------------------
CALL sp_sync_message_stats();
```

user_id	full_name	username	total_messages_s...	total_messages_receiv...	total_activ...	last_updated
1	Иван Петров	ivan_p	4	3	7	2026-03-25 06:23:41
2	Мария Иванова	maria_i	2	3	5	2026-03-25 06:23:41
3	Георги Стоянов	gosho99	3	1	4	2026-03-25 06:23:41
4	Петя Димитрова	petya_d	1	2	3	2026-03-25 06:23:41
5	Стефан Николов	stefan_n	1	2	3	2026-03-25 06:23:41